

Упражнение 1

-

Упражнение 2

1. Изтегляне (<https://visualstudio.microsoft.com/downloads/>) и инсталиране/модифициране на Visual Studio Community - да се изберат пакети: **ASP.NET and web development** (за този пакет във Visual Studio Community 2026 в дясната страна трябва да се сложи отметка и на **.NET Framework project and item templates**, ако не е видимо в дясно, може да се открие в Individual Components) и **Data storage and processing**.
2. Създаване на ново Web приложение – C#, WEB -> **ASP.NET Web Application (.NET Framework)** -> име на проекта, място за съхранение на проекта, избор на версия на .NET. - > Empty
3. Добавяне на уеб форма - **Project -> Add New Item -> Web Form**

Задача 0. Да се добави нов бутон. В текстовата кутия се въвежда едно цяло трицифрено число (приемаме, че числото не съдържа цифра 0). С цифрите на въведеното число програмата да образува най-голямото и най-малкото трицифрено число. Резултатът да се извежда в етикета.

```
protected void Button1_Click(object sender, EventArgs e)
{
    int n, a, b, c, t, min, max;
    n = int.Parse(TextBox1.Text);
    a = n / 100;
    b = (n / 10) % 10;
    c = n % 10;
    // a < b < c
    if (a > b)
    {
        t = a;
        a = b;
        b = t;
    }
    if (b > c)
    {
        t = b;
        b = c;
        c = t;
    }
    if (a > b)
    {
        t = a;
        a = b;
        b = t;
    }

    min = a * 100 + b * 10 + c;
    max = c * 100 + b * 10 + a;

    Label1.Text = "min = " + min + "<br />" + "max = " + max;
}
```

Задача 1. Да се добавят 3 текстови кутии, един бутон и един етикет. След като във всяка от текстовите кутии се въведе цяло число (положително, отрицателно или 0) и след натискане на бутона, в етикета да се изведе съобщение, даващо информация за всяко от числата дали е положително, отрицателно или нула.

```
protected void Button1_Click(object sender, EventArgs e)
{
    int a, b, c;
    a = int.Parse(TextBox1.Text);
    b = int.Parse(TextBox2.Text);
    c = int.Parse(TextBox3.Text);
    string resultat = "";

    resultat = "Първото число е ";
    if(a>0)
    {
        resultat = resultat + "положително"; // resultat += "положително";
    }
    else if (a<0)
    {
        resultat = resultat + "отрицателно"; // resultat += "отрицателно";
    }
    else
    {
        resultat = resultat + "нула"; // resultat += "нула";
    }

    resultat += "<br />Второто число е ";
    if (b > 0)
    {
        resultat = resultat + "положително"; // resultat += "положително";
    }
    else if (b < 0)
    {
        resultat = resultat + "отрицателно"; // resultat += "отрицателно";
    }
    else
    {
        resultat = resultat + "нула"; // resultat += "нула";
    }

    resultat += "<br />Третото число е ";
    if (c > 0)
    {
        resultat = resultat + "положително"; // resultat += "положително";
    }
    else if (c < 0)
    {
        resultat = resultat + "отрицателно"; // resultat += "отрицателно";
    }
    else
    {
        resultat = resultat + "нула"; // resultat += "нула";
    }

    Label1.Text = resultat;
```

```
}
```

Задача 2. Задача 1 да се реализира с помощта на потребителска функция, в която реално ще се извършва проверката дали числото е положително, отрицателно или 0.

```
string proverka(int n)
{
    if (n > 0)
    {
        return "положително";
    }
    else if (n < 0)
    {
        return "отрицателно";
    }
    else
    {
        return "нула";
    }
}

protected void Button2_Click(object sender, EventArgs e)
{
    int a, b, c;
    a = int.Parse(TextBox1.Text);
    b = int.Parse(TextBox2.Text);
    c = int.Parse(TextBox3.Text);

    Label1.Text = "Първото число е " + proverka(a) + "<br /> Второто число е " + proverka(b) + "<br /> Третото число е " + proverka(c);
}
```

Задача 3. Да се добавят 2 текстови кутии и един бутон. В първата текстова кутия да се въведе цяло положително число N. При натискане на бутона в етикет да се изведе сумата на числата от 1 до N.

Допълнение 1: Сумата на числата от 1 до N, които са кратни на любимото Ви число.

Допълнение 2: Да се въвеждат две числа a и b и програмата да намира и извежда сумата на числата от a до b вкл.

```
protected void Button3_Click(object sender, EventArgs e)
{
    int N = int.Parse(TextBox1.Text);
    int sum = 0;
    for(int i = 1; i <= N; i++)
    {
        sum = sum + i;
    }
    Label1.Text = sum.ToString();
}

protected void Button3_Click(object sender, EventArgs e)
{
    int N = int.Parse(TextBox1.Text);
    int k = 7; // любимото число

    int sum = 0;
    for(int i = 1; i <= N; i++)
```

```

    {
        if(i % k == 0)
        {
            sum = sum + i;
        }
    }
    Label1.Text = sum.ToString();
}

```

`protected void Button3_Click(object sender, EventArgs e) //сумата на числата от a до b вкл.`

```

{
    int a = int.Parse(TextBox1.Text);
    int b = int.Parse(TextBox2.Text);

    int sum = 0;

    if (a > b)
    {
        a = a + b;
        b = a - b;
        a = a - b;
    }

    for(int i = a; i <= b; i++)
    {
        sum = sum + i;
    }

    Label1.Text = sum.ToString();
}

```

Задача 4. Да се напише програма, която извежда в обратен ред цифрите на цяло положително число - първо цифрата на единиците, след това цифрата на десетиците и т.н. (с помощта на оператор **do while**)

Задача 5. На коктейла след конференцията на катедра “Информатика” ще присъстват N участници (въвежда се в текстова кутия). Заведението разполага с маси от по 3, 4 и 6 места. Напишете програма, която да определи колко маси трябва от всеки вид, така че да бъдат настанени всички участници. Изведете в етикет всички възможни комбинации на отделни редове (използва се “
”).

```

protected void Button5_Click(object sender, EventArgs e)
{
    int n = int.Parse(TextBox1.Text);
    int a, b, c;
    string result = "";

    for(a=0; a <= n/3; a++)
    {
        for (b = 0; b <= n/4; b++)
        {
            for (c = 0; c <= n/6; c++)
            {
                if ((3*a + 4*b + 6*c) == n)

```

```

        {
            result += a + ", " + b + ", " + c + "<br />";
        }
    }
}
Label1.Text = result;
}

```

Задача 6. Да се отпечата в етикет (на отделни редове "
") с помощта на оператор **for** всички тройки от цели положителни числа **a**, **b**, **c**, по-малки от 10, за които е изпълнено, че

$$a^2 + b^2 = c^2$$

```

protected void Button6_Click(object sender, EventArgs e)
{
    int a, b, c;
    Label1.Text = "";
    for (a = 1; a <= 10; a++)
    {
        for (b = a; b <= 10; b++)
        {
            for (c = b; c <= 10; c++)
            {
                if (a * a + b * b == c * c)
                {
                    Label1.Text += a + " " + b + " " + c + "<br />";
                }
            }
        }
    }
}

```

Задача 7. Да се добави нов бутон. В текстовата кутия се въвежда едно цяло трицифрено число. Програмата да проверява дали числото е кратно на сумата от цифрите си и да извежда подходящо съобщение в етикета.

Допълнение: Програмата да проверява дали числото е кратно на произведението от цифрите си.

```

private void button7_Click(object sender, EventArgs e)
{
    int n = int.Parse(textBox1.Text);
    int a, b, c, s, p;
    a = n / 100;
    b = (n / 10) % 10;
    c = n % 10;
    s = a + b + c;
    p = a * b * c;
    if (n % s == 0)
    {
        label1.Text = "Кратно на сумата! ";
    }
    else
    {

```

```

        label1.Text = "Не е кратно на сумата! ";
    }
    if (p != 0)
    {
        if (n % p == 0)
        {
            label1.Text += "Кратно на произв.!!";
        }
        else
        {
            label1.Text += "Не е кратно на произв.!!";
        }
    }
    else
    {
        label1.Text += "Не е кратно на произв.!!";
    }
}

```

Упражнение 3

-

Упражнение 4

Задача 1. Да се добавят нов бутон, етикет и три текстови кутии. Приемаме, че в първите две текстови кутии се въвеждат цели числа, а в третата аритметична операция: +, -, *, /, или ^ (степенуване - първото число на степен второто). При натискане на бутона да се прави проверка за въведената операция и тя да се прилага върху числата, въведени в първите две текстови кутии. Ако в третата текстова кутия се въведе символ, различен от посочените знаци, в етикета да се изведе съобщение "Невалидна операция". Резултатът се извежда в етикета.

```

private void button1_Click(object sender, EventArgs e)
{
    int a, b, rez = 1;
    a = int.Parse(TextBox1.Text);
    b = int.Parse(TextBox2.Text);
    string op = TextBox3.Text;
    bool x = true;
    bool valid = true; //ако е въведен невалиден символ за операция - присвояваме стойност false
    switch (op)
    {
        case "+":
            rez = a + b;
            break;
        case "-":
            rez = a - b;
            break;
        case "*":
            rez = a * b;
            break;
        case "/":
            if (b != 0)
            {
                rez = a / b;
            }

```

```

        else
        {
            x = false;
        }
        break;
    case '^':
        for(int i=1; i <=b; i++)
        {
            rez = rez * a;
        }
        break;
    default:
        valid = false;
        break;
}
if (!valid)
{
    Label1.Text = "Невалидна операция!";
}
else if (!x)
{
    Label1.Text = "Не може да се дели на 0!";
}
Label1.Text = rez.ToString();
}

```

Задача 2. Изчисляване на индекс на телесна маса (ИТМ) – в две текстови кутии се въвеждат съответно тегло в кг и височина. Индексът на телесна маса се изчислява по формулата:

тегло в кг / (височина в м)²

ИТМ под 18.5 kg/m² – поднормено телесно тегло

ИТМ 18.5 - 24.9 kg/m² – нормално телесно тегло

ИТМ 25 - 29.9 kg/m² – наднормено телесно тегло

ИТМ 30 - 34.9 kg/m² – затлъстяване I степен

ИТМ 35 - 39.9 kg/m² – затлъстяване II степен

ИТМ ≥ 40kg/m² – затлъстяване III степен (високостепенно затлъстяване)

Програмата да извежда съобщение с здравословното състояние на потребителя според ИТМ.

Ако ИТМ е над 24.9, в етикет да се извежда съобщение с изчислено здравословно тегло (при въведената височина да му гарантира ИТМ 24.9).

```

protected void Button2_Click(object sender, EventArgs e)
{
    double t, r, itm;
    t = double.Parse(TextBox1.Text);
    r = double.Parse(TextBox2.Text);

    string rez = "";

    if (r > 2.5)
    {
        r = r / 100;
    }

    itm = t / (r * r);

    if (itm < 18.5)
    {

```

```

        rez = "поднормено телесно тегло";
    }
    else if(itm < 25)
    {
        rez = "нормално телесно тегло";
    }
    else if(itm < 30)
    {
        rez = "наднормено телесно тегло";
    }
    else if (itm < 35)
    {
        rez = "затлъстяване I степен";
    }
    else if (itm < 40)
    {
        rez = "затлъстяване II степен";
    }
    else
    {
        rez = "затлъстяване III степен";
    }
    rez += " ИТМ = " + itm.ToString("f1");

    if (itm > 24.9)
    {
        rez = rez + " Препоръчително тегло: " + (24.9*r*r).ToString("f0") + " кг";
    }
    Label1.Text = rez;
}

```

Задача 3. При натискане на бутон да се създава масив от 5 цели числа. Програмата да извършва следните действия: да запълни масива със случайни числа в интервала от 2 до 6 вкл.; в етикет да се извеждат третия елемент от масива; елемента на масива с най-голяма стойност.

Допълнение: Да се напише потребителска функция, която намира сумата от цифрите на подадено като параметър цяло число. Функцията намира и връща като резултат сумата от цифрите на числото. С помощта на функцията да се пресметне сбора от цифрите на елементите в масива.

```

Random chislo = new Random();

protected void Button3_Click(object sender, EventArgs e)
{
    int[] m = new int[5];
    Label1.Text = "";
    for(int i = 0; i < m.Length; i++)
    {
        m[i] = chislo.Next(2, 7);
        Label1.Text += m[i] + " ";
        if (i == 2)
        {
            Label2.Text = m[i].ToString();
        }
    }

    int max = m[0], min = m[0];
    for(int i = 1; i < m.Length; i++)

```



```

    {
        if (m[i] > max)
        {
            max = m[i];
        }
        if(m[i] < min)
        {
            min= m[i];
        }
    }

    Label2.Text += " Максималната ст-ст е " + max;
}

```

Задача 4. При натискане на бутон да се създава масив с брой на елементите, равен на въведено в текстовя кутия цяло число. Масивът да се запълни със случайни числа в интервала от 2 до 6 вкл. Стойностите на масива да се изведат в етикет. Да се прави проверка дали масивът е симетричен и в етикет да се изписва “Да”, ако е симетричен и “Не”, ако не е такъв.

```

Random r=new Random();

protected void Button1_Click(object sender, EventArgs e)
{
    int n = int.Parse(TextBox1.Text);
    int[] m = new int[n];
    for(int i=0; i<m.Length; i++)
    {
        m[i] = r.Next(2, 7);
        Label1.Text = Label1.Text + m[i] + " ";
    }

    bool sim = true;
    for (int i = 0; i < n / 2; i++)
    {
        if (m[i] == m[n - 1 - i])
        {
            sim = false;
            break;
        }
    }

    if (sim)
    {
        Label1.Text += " Симетричен";
    }
    else
    {
        Label1.Text += " Не е симетричен";
    }
}

```

Задача 5. Да се създаде масив, в който се пазят данни за оценките на студент от 3-ти курс, получени на изпитите по време на обучението му до момента – 20 оценки. Да се намери средната стойност на резултатите. Да се изчисли разликата между най-ниския и най-високия резултат. Да се изведат медианата и модата на резултатите на студента.
Примерни данни: { 6, 5, 4, 3, 6, 6, 5, 4, 5, 4, 4 , 3, 5, 3, 4, 6, 6, 3, 5, 4}

Упражнение 5

Задача 1. Да се добавят 2 текстови кутии, 1 бутон и 2 етикета (първоначално етикетите да са скрити). В текстовите кутии ще се въвеждат количество гориво (литри) и цена на гориво (лв./литър). При натискане на бутона в първия етикет да се изведе общата стойност в лева. Във втория етикет да се изведе общата стойност в евро (курс 1.95583). Да се направи необходимата проверка за наличие на стойности в текстовите кутии.

Допълнение: Празните текстови кутии да се оцветяват в червено и в единия етикет да се изписва подходящ текст (например „Въведете литри и цена на литър.“).

```
protected void Button1_Click(object sender, EventArgs e)
```

```
{
    TextBox1.BackColor = Color.Empty;
    TextBox2.BackColor = Color.Empty;

    double l, cena, suma_lv, evro;

    if (TextBox1.Text == "" && TextBox2.Text == "")
    {
        TextBox1.BackColor = Color.Red;
        TextBox2.BackColor = Color.Red;
        Label1.Text = "Не са въведени литри";
        Label2.Text = "Не е въведена цена";
    }
    else if (TextBox1.Text == "")
    {
        TextBox1.BackColor = Color.Red;
        Label1.Text = "Не са въведени литри";
        Label2.Text = "";
    }
    else if (TextBox2.Text == "")
    {
        TextBox2.BackColor = Color.Red;
        Label2.Text = "Не е въведена цена";
        Label1.Text = "";
    }
    else
    {
        l = double.Parse(TextBox1.Text);
        cena = double.Parse(TextBox2.Text);
        suma_lv = l * cena;
        evro = suma_lv / 1.95583;

        Label1.Text = "Сума в лв. " + suma_lv.ToString("f2");
        Label2.Text = "Сума в евро " + evro.ToString("f2");
    }

    Label1.Visible = true;
    Label2.Visible = true;
}
```

Задача 2. Да се добавят *LinkButton* с текст „Провери маршрут“ и *HyperLink* (първоначално да е скрит). Да се направи така, че при натискане на *LinkButton* *HyperLink*-ът да стане видим и да сочи към адрес <https://www.google.com/maps> Адресът да се отваря в нов прозорец/таб.

Допълнение: Да се добави още една текстова кутия, в която приемаме, че потребителят въвежда линк (например конкретен маршрут от Google Maps) и LinkButton-ът да настройва HyperLink-а да сочи към адреса, написан в текстовата кутия.

```
protected void LinkButton1_Click(object sender, EventArgs e)
{
    //HyperLink1.Text = "www.google.com/maps";
    //HyperLink1.NavigateUrl = "https://www.google.com/maps";

    HyperLink1.Text = TextBox1.Text;
    HyperLink1.NavigateUrl = "https://" + TextBox1.Text;

    HyperLink1.Target="_blank";

    HyperLink1.Visible = true;
}
```

Задача 3. Да се добави *Image* и картинка, свързана с пътуване (кола, карта, път, гориво, компас), която да се показва в Image-а. Може от проекта - *десен бутон към проекта и Add Existing Item* или *директно от интернет с линк*.

Задача 4. Да се добавят текстова кутия, *DropDownList* и *ImageButton* с картинка (фон) по Ваш избор - може от проекта или директно от интернет с линк.

При натискане на ImageButton-а в DropDownList-а да се добавя въведения в текстовата кутия текст - да приемем, че се въвежда име на туристически обект в България. Да се провери дали новият елемент вече не е наличен в DropDownList –а и ако е така, да не се въвежда повторно в DropDownList-а.

Допълнение: Да се добави още една текстова кутия, като приемаме, че в нея се въвежда разстояние в км от гр. Варна до туристическия обект и при натискане на ImageButton-а в DropDownList-а да се добавят както името на туристическия обект, така и разстоянието до него - като стойност на елемента.

Например:

Царевец - 230 км

Ягодинска пещера - 460 км

Връх Снежанка - 430 км

Боженци - 270 км

Сребърна - 150 км

Общо 1540 км

```
protected void ImageButton1_Click(object sender, ImageClickEventArgs e)
{
    Label1.Text = "";
    // DropDownList1.Items.Add(TextBox1.Text);

    //DropDownList1.Items.Add(new ListItem(TextBox1.Text, TextBox2.Text));
    bool find = false;

    ListItem li = new ListItem(TextBox1.Text, TextBox2.Text);

    if (!DropDownList1.Items.Contains(li))
    {
        DropDownList1.Items.Add(li);
    }
    else
    {
        Label1.Text = "Вече го има!";
    }
}
```

```

        Label1.Visible = true;
    }

    /*
    for (int i = 0; i < DropDownList1.Items.Count; i++)
    {
        if (TextBox1.Text == DropDownList1.Items[i].Text)
        {
            find = true;
            break;
        }
    }
    */
    /*
    foreach(ListItem item in DropDownList1.Items)
    {
        if (TextBox1.Text == item.Text)
        {
            find = true;
            break;
        }
    }

    // if (find == false)
    if(!find)
    {
        ListItem li = new ListItem(TextBox1.Text, TextBox2.Text);
        DropDownList1.Items.Add(li);
    }
    else
    {
        Label1.Text = "Вече го има!";
        Label1.Visible = true;
    }
    */
}

```

Задача 5. Да се добавят етикет и ImageButton, който да изчислява общото разстояние в км от гр. Варна до всички обекти в DropDownList-a. В етикета да се извежда удвоената стойност на тази сума.

```

protected void Button2_Click(object sender, EventArgs e)
{
    int sum = 0;

    for (int i = 0; i < DropDownList1.Items.Count; i++)
    {
        sum = sum + int.Parse(DropDownList1.Items[i].Value);
    }

    Label2.Text = (sum*2).ToString();
    Label2.Visible = true;
}

```

Задача 6. Да се добави текстова кутия. При избор на елемент от DropDownList-a името на избрания обект да се визуализира в текстовата кутия.

Допълнение: Да се визуализира и разстоянието до този обект.

AutoPostBack - true

```

protected void DropDownList1_SelectedIndexChanged(object sender, EventArgs e)
{

```

```

Label1.Text = DropDownList1.SelectedItem + " - " + DropDownList1.SelectedValue;
Label1.Visible = true;
}

```

Задача 7. Списъкът с туристически обекти (елементите в DropDownList-a) да се сортира по име на обект от А до Я.

```

protected void Button2_Click(object sender, EventArgs e)
{
    string t, v;
    bool sort;

    for (int k = 1; k < DropDownList1.Items.Count; k++)
    {
        sort = true;

        for (int i = 1; i < DropDownList1.Items.Count - 1; i++)
        {
            if (string.Compare(DropDownList1.Items[i].Text, DropDownList1.Items[i+1].Text) == 1)
            {
                sort = false;
                t = DropDownList1.Items[i].Text;
                DropDownList1.Items[i].Text = DropDownList1.Items[i+1].Text;
                DropDownList1.Items[i+1].Text = t;

                v = DropDownList1.Items[i].Value;
                DropDownList1.Items[i].Value = DropDownList1.Items[i + 1].Value;
                DropDownList1.Items[i + 1].Value = v;
            }
        }
        if (sort)
        {
            break;
        }
    }
}

```

Упражнение 6

Задача 1. Да се добави *CheckBox* и бутон. При натискане на бутона съдържанието на текстовата кутия да се добавя в *DropDownList*-а само, ако *CheckBox*-ът е избран.

Добавка: Да се добави още един *CheckBox*. Ако и той е избран, да се прави проверка дали стойността в текстовата кутия се съдържа в *DropDownList*-а. Ако се съдържа - да не се добавя отново. Ако вторият *CheckBox* не е избран, то в *DropDownList*-а могат да се добавят еднакви стойности. (*Забележка: Идеята на задачата е да се предложи решение на задача 4 от Упр. 3 с използване на CheckBox*).

```

protected void Button1_Click(object sender, EventArgs e)
{
    if (CheckBox1.Checked)
    {
        if (TextBox1.Text != "" && TextBox2.Text != "")
        {
            ListItem item = new ListItem(TextBox1.Text, TextBox2.Text);

            if (!DropDownList1.Items.Contains(item))
            {

```

```

        DropDownList1.Items.Add(item);
    }
    else
    {
        Label1.Text = "Има го";
        Label1.Visible = true;
        TextBox1.Text = "";
        TextBox2.Text = "";
    }
}
else
{
    Label1.Text = "Попълнете!";
    Label1.Visible = true;
}
}
}

```

Задача 2. Да се добави *ListBox* и в него да се изведат елементите от *DropDownList*-а: пореден номер на елемента (на обекта в списъка), име на обект и км.

```

protected void Button3_Click(object sender, EventArgs e)
{
    ListBox1.Items.Clear();

    for (int i = 1; i < DropDownList1.Items.Count; i++)
    {
        ListBox1.Items.Add(i + DropDownList1.Items[i].Text + " - " + DropDownList1.Items[i].Value);
    }
}

```

Задача 3. Да се добавят 4 *CheckBox*-а. Потребителят въвежда обща стойност на почивката в текстова кутия и маркира чекбоксове според това кои отстъпки важат за него:

- ☐ Карта за лоялност (-5%)
- ☐ Покупка над 100 евро (-10%)
- ☐ Специална празнична промоция (-15%)
- ☐ Промокод (-20%)

При натискане на бутон, програмата да проверява кои отстъпки са избрани и да ги приложи към общата сума. Крайната стойност след всички намаления да се извежда в етикет.

Допълнение: Ако няма избрана нито една отстъпка, да се извежда в етикета: „Няма приложена отстъпка“. Ако общата сума е под 50 евро, не се прилага никаква отстъпка, дори ако има маркирани чекбоксове. Ако са избрани всички отстъпки, програмата да дава максимална отстъпка 40% вместо 50% (за да се избегнат твърде големи намаления).

```

protected void Button1_Click(object sender, EventArgs e)
{
    double suma = double.Parse(TextBox1.Text);
    int p = 0;

    if (suma < 50)
    {
        Label1.Text = "Похарчи още " + (50 - suma) + " евро";
    }
    else
    {

```

```

        if (CheckBox1.Checked)
        {
            p = p + 5;
        }
        if (CheckBox2.Checked)
        {
            p = p + 10;
        }
        if (CheckBox3.Checked)
        {
            p = p + 15;
        }
        if (CheckBox4.Checked)
        {
            p = p + 20;
        }
        if (p > 40)
        {
            p = 40;
        }
        Label1.Text = p + "% " + (suma - suma*p/100) + " евро";
    }
}

```

Задача 4. Да се добави и *CheckBoxList*, който да се използва за решаване на задача 3. За стойности (свойство *Value*) на елементите да се зададат съответно процентите отстъпка - добавяме 4-те възможности от зад. 3 статично от *Edit Items* на *CheckBoxList-a*

```

protected void Button2_Click(object sender, EventArgs e)
{
    double suma = double.Parse(TextBox1.Text);
    int p = 0;

    foreach (ListItem item in CheckBoxList1.Items)
    {
        if (item.Selected)
        {
            p = p + int.Parse(item.Value);
        }
    }
    if (p > 40)
    {
        p = 40;
    }
    Label1.Text = p + "% " + (suma - suma * p / 100) + " евро";
}

```

Задача 5. Да се добавят текстова кутия и 3 радио бутона с имена *red*, *green*, *blue*. Да се добави нов обикновен бутон, който да сменя цвета на фона на текстовата кутия в зависимост от избрания радио бутон.

```

protected void Button3_Click(object sender, EventArgs e)
{
    if (rb_red.Checked)
    {
        TextBox1.BackColor = Color.Red;
    }
}

```

```

    }
    else if (rb_green.Checked)
    {
        TextBox1.BackColor = Color.Green;
    }
    else if (rb_blue.Checked)
    {
        TextBox1.BackColor = Color.Blue;
    }
    else
    {
        TextBox1.BackColor = Color.Empty;
    }
}

```

Задача 6. Да се добави един `RadioButtonList`, един обикновен бутон и една текстова кутия. При натискане на бутона – той да добавя нови елементи (радио бутони) в списъка, съответно с имена “rb”+ поредния номер на радиобутона.

```

protected void Button4_Click(object sender, EventArgs e)
{
    int i = RadioButtonList1.Items.Count + 1;
    RadioButtonList1.Items.Add("rb " + i);
}

```

Допълнение: При избор на някой от радио бутоните от списъка в текстовото поле да се изпише кой е избран.

AutoPostBack true

```

protected void RadioButtonList1_SelectedIndexChanged(object sender, EventArgs e)
{
    Label1.Text = RadioButtonList1.SelectedItem.ToString();
}

```

Задача 7. Да се добавят към проекта 4 изображения за 4-те сезона и един *Image*. При избор на конкретен радиобутон от *RadioButtonList* (с опции пролет, лято, есен, зима) да се показва изображението, съответстващо на избрания сезон.

```

protected void RadioButtonList2_SelectedIndexChanged(object sender, EventArgs e)
{
    Image1.ImageUrl = "image" + RadioButtonList2.SelectedValue + ".jpg";
}

```

Упражнение 7

Задача 1. Приложение за изчисляване на дължимия данък за МПС 2026 г. за община Варна. Да се добавят *текстова кутия*, два *радиобутона* (kw и к.с.), *етикети*, 2 *DropDownList*, *RadioButtonList*, *Image* и *бутон*. При натискане на бутона да се изчислява дължимия данък за лек автомобил за град Варна.

Потребителят избира от *DropDownList* с въведени имената на няколко областни града област по постоянен адрес / седалище на собственика. При избор на град от списъка в *Image* да се показва герба на избрания град. Ако потребителят избере град, различен от Варна, то в етикет да се изписва съобщение, че системата е в процес на разработка и работи само за област Варна.

Мощността на двигателя се въвежда в текстовата кутия и за формулата трябва да е в *kw*. При зареждане на страницата да е избран радиобутон *kw*. Ако потребителят избере да въвежда мощността в конски сили – избрал е радиобутон *к.с.*, то да се преобразуват конските сили в *kw*: **1 kw = 1.341 конски сили**

Частта от стойността на данъка в зависимост от **мощността** на двигателя се определя, както следва:

До 55 kw включително – 0.95 лв./ 1 kw

Над 55 kw до 74 kw включително – 1.28 лв./ 1 kw

Над 74 kw до 110 kw включително – 1.57 лв. / 1 kw

Над 110 kw до 150 kw включително – 1.76 лв. / 1 kw

Над 150 kw до 245 kw включително – 1.90 лв. / 1 kw

Над 245 kw – 2.10 лв. / 1 kw

КГП е коригиращ коефициент за годината на производство на автомобила.

Възрастта (в години) на автомобила се избира от *RadioButtonList* със следните елементи:

Над 20 години – 1.1

От 16 до 20 години – 1.0

От 11 до 15 години – 1.3

От 6 до 10 години– 1.5

От 0 до 5 години– 2.3

Стойностите показват коригиращия коефициент за съответната възраст.

Екологичният компонент се определя в зависимост от екологичната категория на автомобила.

Екологичната категория се избира от *DropDownList* със следните стойности:

Екологична категория	Коефициент
без екологична категория Евро 1 Евро 2	1.10
Евро 3	1.00
Евро 4	0.8
Евро 5	0.6
Евро 6 EEV	0.4

На потребителя да се изведе както стойността на дължимия данък, така и съответните стойности на коефициентите според въведените от него данни - в *BulletedList*.

Да се покаже и хипервръзка с текст "НАРЕДБА на ОбС Варна", която да води към <https://varna.obshtini.bg/doc/345772>

```
protected void Button1_Click(object sender, EventArgs e)
{
    BulletedList1.Visible = false;
```

```

Label1.Visible = false;
HyperLink1.Visible = false;

if(DropDownList1.SelectedValue == "9000")
{
    double kw = double.Parse(TextBox1.Text);
    double tax, ek, age, price;

    if (ks.Checked)
    {
        kw = Math.Round( kw / 1.341, 0);
    }

    if (kw <= 55)
    {
        price = 0.95;
    }
    else if (kw <= 74)
    {
        price = 1.28;
    }
    else if (kw <= 110)
    {
        price = 1.57;
    }
    else if (kw <= 150)
    {
        price = 1.76;
    }
    else if (kw <= 245)
    {
        price = 1.90;
    }
    else
    {
        price = 2.10;
    }

    ek = double.Parse(DropDownList2.SelectedValue);
    age = double.Parse(RadioButtonList1.SelectedValue);

    tax = (kw*price) * ek * age;

    BulletedList1.Items.Clear();

    BulletedList1.Items.Add("Ставка за киловат " + price);
    BulletedList1.Items.Add("Коефициент за ЕК " + ek);
    BulletedList1.Items.Add("Коефициент за възраст " + age);

    Label1.Text = "Дължим данък " + tax.ToString("f2") + " лв. / " + (tax/1.95583).ToString("f2") + " евро";

    BulletedList1.Visible = true;
    Label1.Visible = true;

    HyperLink1.Text = "НАРЕДБА на ОПС Варна";
    HyperLink1.NavigateUrl = "https://varna.obshtini.bg/doc/345772";
    HyperLink1.Target = "_blank";
    HyperLink1.Visible = true;
}
else

```

```

{
    Label1.Text = "Сайтът работи само за Варна.....";
    Label1.Visible = true;
}
}

```

Задача 2. Да се добави бутон. При натискане на бутона, съдържанието на *RadiobuttonList*-а с възрастта в години да се представя в таблица с две колони: интервал и коефициент за съответния интервал. Съдържанието на *DropDownlist*-а с екологичните категории и съответните коефициенти да се извежда в нова таблица с три колони – пореден номер, екологична категория и коефициент.

```

protected void Button1_Click(object sender, EventArgs e)
{
    for(int r = 0; r < 5; r++)
    {
        TableRow red = new TableRow();
        Table1.Rows.Add(red);

        for(int c = 1; c <= 20; c++)
        {
            TableCell cc = new TableCell();
            cc.Text = (r+1) + ", " + c;
            // red.Cells.Add(cc);
            Table1.Rows[r].Cells.Add(cc);
        }
    }

    /*
    TableRow r1 = new TableRow();
    TableRow r2 = new TableRow();

    TableCell c1= new TableCell();
    TableCell c2 = new TableCell();
    TableCell c3 = new TableCell();
    TableCell c4 = new TableCell();

    c1.Text = "1,1";
    c2.Text = "1,2";
    c3.Text = "2,1";
    c4.Text = "2,2";

    r1.Cells.Add(c1);
    r1.Cells.Add(c2);
    r2.Cells.Add(c3);
    r2.Cells.Add(c4);

    Table1.Rows.Add(r1);
    Table1.Rows.Add(r2);
    */
}

protected void Button2_Click(object sender, EventArgs e)
{
    TableRow rh = new TableRow();
    Table2.Rows.Add(rh);

    TableCell c1 = new TableCell();
    TableCell c2 = new TableCell();
    TableCell c3 = new TableCell();

```

```

c1.Text = "Номер";
c2.Text = "Община";
c3.Text = "ПК";

Table2.Rows[0].Cells.Add(c1);
Table2.Rows[0].Cells.Add(c2);
Table2.Rows[0].Cells.Add(c3);

for (int i = 0; i < DropDownList1.Items.Count; i++)
{
    TableRow tr = new TableRow();
    Table2.Rows.Add(tr);

    TableCell num = new TableCell();
    TableCell ob = new TableCell();
    TableCell pk = new TableCell();

    num.Text = (i+1).ToString();
    pk.Text = DropDownList1.Items[i].Value;
    ob.Text = DropDownList1.Items[i].Text;

    Table2.Rows[i + 1].Cells.Add(num);
    Table2.Rows[i + 1].Cells.Add(ob);
    Table2.Rows[i + 1].Cells.Add(pk);
}
}

```

Задача 3. Да се добави календар и при избор на дата в календара, ако тя е преди 1 април, то се начислява 5% отстъпка за дължимия данък.

Допълнение: Да се добави текстова кутия. При избор на дата от календара тя да се изпише в текстовата кутия във формат *ден.месец.година г.* (например: 07.04.2026 г.)

```

protected void Button3_Click(object sender, EventArgs e)
{
    Random random = new Random();
    DateTime d = new DateTime(2026, 3, random.Next(1,32));
    Calendar1.SelectedDate = d;

    DateTime dnes = DateTime.Today;
    DateTime srok = new DateTime(2026, 4, 1);

    if (dnes < srok)
    {
        Label1.Text = "5% отстъпка. Остават " + (srok-dnes).Days + " дни";
    }
    else
    {
        Label1.Text = "Няма отстъпка";
    }
}

protected void Calendar1_SelectionChanged(object sender, EventArgs e)
{
    DateTime srok = new DateTime(2026, 4, 1);
    DateTime izbrana = Calendar1.SelectedDate;

    if (izbrana < srok)
    {

```

```

        Label2.Text = "5% отстъпка. Остават " + (srok - izbrana).Days + " дни";
    }
    else
    {
        Label2.Text = "Няма отстъпка";
    }
}

```

Задача 4. Да се добавят календар и един бутон. При кликване върху бутона в календара да се маркират едновременно две дати (7 април 2026 г. и 19 май 2026 г. – датите на контролните работи).

```

protected void Button3_Click(object sender, EventArgs e)
{
    DateTime kr1 = new DateTime(2026,4,7);
    DateTime kr2 = new DateTime(2026, 5, 19);
    Calendar1.SelectedDates.Add(kr1);
    Calendar1.SelectedDates.Add(kr2);
}

```

Упражнение 8

Контролна работа 1

Упражнение 9

1. Създаване на ново *Blazer Web App* приложение. Запазване на стандартни настройки.
2. Разглеждане на някои от страниците в папка Components -> Pages.
 - В Home.razor - @page, <PageTitle>
 - В Counter.razor – добавен е бутон и е прихванато събитието Click
3. В *Home* страницата да се добави като компонент *Counter*.

```

@page "/"
<PageTitle>Home</PageTitle>
<h1>Hello, world!</h1>
Welcome to your new app.
<Counter />

```

4. Да се добави параметър в *Counter*, на който да се предаде стойност от *Home* - начална стойност на брояча.

```

@page "/"
<PageTitle>Home</PageTitle>
<h1>Hello, world!</h1>
Welcome to your new app.
<Counter CurrentCount="10" />

```

В Counter.razor

```

@page "/counter"
@rendermode InteractiveServer
<PageTitle>Counter</PageTitle>
<h1>Counter</h1>
<p role="status">Current count: @CurrentCount</p>
<button class="btn btn-primary" @onclick="IncrementCount">Click me</button>

```

```

@code {

```

```

[Parameter]
public int CurrentCount { get; set; } = 0;

private void IncrementCount()
{
    CurrentCount++;
}
}

```

5. Да се добави в *Counter* и параметър, който да определя стъпката, с която да се променя брояча.

В *Counter.razor* добавяме:

```

[Parameter]
public int Step { get; set; } = 1;

```

А в *Home.razor*:

```

<Counter CurrentCount="10" Step="5" />

```

6. Да се добави нов компонент към проекта:
Project -> Add New Item -> Razor Component (Pages -> Add -> Razor Component)
и да се включи в *Home*.
7. Да се добави нова страница към проекта *Pages -> Add -> Razor Component* и добавяне на `@page`

Stranica1.razor

```

@page "/str1"
<PageTitle>Нова страница</PageTitle>
<h3>Добре дошли!</h3>

```

8. Да се добави новата страница към менюто.

В компонента *NavMenu.razor* добавяме:

```

<div class="nav-item px-3">
    <NavLink class="nav-link" href="str1">
        <span class="bi bi-list-nested-nav-menu" aria-hidden="true"></span> Моята страница
    </NavLink>
</div>

```

9. Да се добави текстова кутия в новата страница.
10. В новата страница да се добави компонент *Counter*, чиято стъпка да се определя от въведена от потребителя стойност в нова текстова кутия.

Задача 0. В новата страница да се добави текстова кутия. При въвеждане на текст в нея, написаното да се показва в параграф. *Добавка:* Написаното в кутията да се извежда в параграфа при всяка промяна на съдържанието на текстовата кутия.

Stranica1.razor

```

@page "/str1"
@rendermode InteractiveServer

```

```

<PageTitle>Нова страница</PageTitle>
<h3>Добре дошли!</h3>

```

```

<input type="text" @bind="my_text" />

```

```

<p>Вие въведохте: @my_text </p>

```

```
@code {  
    private string? my_text;  
}
```

За добавката:

```
<input type="text" @bind="my_text" @bind:event="oninput" />
```

Задача 1. Да се добавят две текстови кутии, един бутон и един етикет. В текстовите кутии следва да се въведе една и съща парола. При натискане на бутона в етикета да се изведе дали паролите съвпадат или не.

Stranica1.razor

```
@page "/"str1"  
@rendermode InteractiveServer
```

```
<PageTitle>Новата ни страница</PageTitle>
```

```
<h3>Моята страница</h3>
```

```
<p>Здравейте!</p>
```

```
<span>Парола:</span>
```

```
<input type="password" @bind="password1" />
```

```
<span>Повтори парола:</span>
```

```
<input type="password" @bind="password2" />
```

```
<button @onclick="proverka">Натисни ме</button>
```

```
<span>Резултат: @resultat</span>
```

```
@code {  
  
private string? password1, password2;  
private string resultat = "";  
  
private void proverka()  
{  
    if (password1 == "" || password2=="")  
    {  
        resultat = "Въведете парола!";  
    }  
    else if (password1 == password2)  
    {  
        resultat = "Паролите съвпадат!";  
    }  
    else  
    {  
        resultat = "Паролите не съвпадат! Въведете наново!";  
    }  
}
```

Задача 2. Да се добави *CheckBox* и 1 текстова кутия. При смяна на стойността на отметката на *CheckBox*-а в текстовата кутия да се изписва "Да" или "Не".

```

.....
<input type="checkbox" @bind="proverka2" @bind:after="proverka3" />Проверка
<input type="text" @bind="yes_no" />

```

```

@code {
.....
private bool proverka2;

private void proverka3()
{
    if (proverka2)
    {
        yes_no = "Да, избран";
    }
    else
    {
        yes_no = "Не, не е избран";
    }
}
}

```

Задача 3. Да се добавят 2 текстови кутии, 1 *checkbox*, 1 бутон и 1 етикет. В текстовите кутии ще се въвеждат количество и цена (реално число). При натискане на бутона в етикета да излезе произведението на количество и цена. Ако *checkbox*-ът е избран, в етикета да се изведе стойността с начислено ДДС 20%.

```

.....
<input type="text" @bind="cena" />

<input type="text" @bind="kol" />

<input type="checkbox" @bind="DDS" />

<span>Дължима сума: @tax</span>
<button @onclick="smetni">Сметни</button>
@code {

    private string? cena, kol;
    private double tax;

    private bool DDS;

    private void smetni()
    {
        tax = double.Parse(cena) * double.Parse(kol);
        if (DDS)
        {
            tax *= 1.2;
        }
    }
}

```

Упражнение 10

Задача 1. Да се създаде нова страница в *Blazor* приложение с име **Hotel**.

В страницата да се добавят:

- 2 текстови кутии: за име на клиент и за брой нощувки
- 3 *checkbox*-а:
 - Закуска (+10 евро на нощувка)

- Вечеря (+20 евро на нощувка)
- Спа (+15 евро на нощувка)
- 1 бутон
- 1 текстов елемент за резултата

При натискане на бутона да се изчислява крайната сума за престоя – цената на 1 нощувка е **120 евро**.

Да се покаже съобщение от вида: „**Клиент Иван Петров дължи за 3 нощувки 390 евро.**“

@page **"/hotel"**

@rendermode InteractiveServer

<h3>Хотел 35</h3>

Име на гост: **<input type="text" @bind="gost" />**

Брой нощувки: **<input type="text" @bind="broi_n" />**

<input type="checkbox" @bind="zak" /> Закуска

<input type="checkbox" @bind="vech" /> Вечеря

<input type="checkbox" @bind="spa" /> Спа

<button @onclick="smetni">Сметни</button>

<p>@gost дължи за @broi_n нощувки @tax евро</p>

```
@code {
    private string? gost;
    private int broi_n, extri = 0, tax;
    private bool zak, vech, spa;
    private void smetni()
    {
        extri = 0;
        if (zak)
        {
            extri += 10;
        }
        if (vech)
        {
            extri += 20;
        }
        if (spa)
        {
            extri += 15;
        }

        tax = broi_n * (120 + extri);
    }
}
```

Задача 2. Да се добави един бутон. При натискане на бутона – той да добавя нови елементи (checkbox-ове) в списък, съответно с имена “chb”+ поредния номер на checkbox-а.

Допълнение: При смяна на отметка в списъка - в нова текстова кутия да се изпише кой от CheckBox-овете е маркиран и кой не е. Например в следния формат - 0 1 0 1.

@page **"/otmetki"**

@rendermode InteractiveServer

Брой отметки <input type="text" @bind="br"/>

Състояние <input type="text" @bind="status" />

@for(int i=0; i < ch_List.Count; i++)

{

int j = i;

<input type="checkbox" @bind="ch_List[j]" @bind:after="StatusChBox" /> @(i+1)

}

<button @onclick="AddChBox">Добави чекбокс</button>

<button @onclick="StatusChBox">Провери избрани/неизбрани </button>

@code {

private int br;

private string status="";

private List<bool> ch_List = new List<bool>();

private void AddChBox()

{

for(int i=0; i<br; i++)

{

ch_List.Add(false);

}

}

private void StatusChBox()

{

status = "";

for(int i=0; i<ch_List.Count; i++)

{

status += ch_List[i] ? 1 : 0;

/*

if (ch_List[i])

{

status += 1;

}

else

{

status += 0;

*/

}

}

}

Задача 3. Задача 1 да се реши като се използва списък с отметки (CheckBoxList).

В Horel.razor добавяме:

.....

@for(int i=0; i < status.Count; i++)

{

int j = i;

<input type="checkbox" @bind="status[j]" @bind:after="presmetni" />

@uslugi[j]

}

<p>Сума @extri</p>

```

@code{
private string? gost;
private int broi_n, extri = 0, tax;
private bool zak, vech, spa;

List<bool> status = new List<bool> { false, false, false };
List<string> услуги = new List<string> {"Закуска", "Вечеря", "Спа" };
List<int> cena = new List<int> {10,20,15 };

private void presmetni()
{
    extri = 0;
    for(int i=0; i<status.Count; i++)
    {
        if (status[i])
        {
            extri += cena[i];
        }
    }
}
}
}

```

Задача 4. Да се добавят 3 радиобутона (номер на стая) - потребителят може да избере само една от трите опции: 101, 102 или 103.

В Horel.razor добавяме:

```

.....
<input type="radio" name="room" @onchange="@ (e => Staya(101))" /> Единична
<input type="radio" name="room" @onchange="@ (e => Staya(102))" /> Двойна
<input type="radio" name="room" @onchange="@ (e => Staya(103))" /> Тройна
<p>Вие избрахте стая номер @r @zabelejka</p>

```

```

@code{
.....

private void Staya (int room)
{
    r = room;
    if (room == 103)
    {
        zabelejka = "Може да се настанят до 6 човека";
    }
}
}
}

```

Упражнение 11

Задача 1. На нова страница да се добавят текстова кутия и 3 радио бутона с имена *red*, *green*, *blue* и да бъдат в една група. Да се добави нов обикновен бутон, който да сменя цвета на фона на TextBox-а в зависимост от избора радио бутон.

```

@page "/upr_11"
@rendermode InteractiveServer

<h3>Цвят</h3>
<input type="text" style="background-color:@cvyat" />
<ul style="list-style-type:none">
@*
    <li> <input type="radio" name="cc" @onchange="@ (e=>Color_name("red"))" />Червен</li>
    <li><input type="radio" name="cc" @onchange="@ (e => Color_name("green"))" />Зелен</li>

```

```

</li><input type="radio" name="cc" @onchange="@ (e => Color_name("blue"))" />Син</li>
*@
<li><input type="radio" name="cc" @onchange="@ (e => Color_name(1))" />Червен</li>
<li><input type="radio" name="cc" @onchange="@ (e => Color_name(2))" />Зелен</li>
<li><input type="radio" name="cc" @onchange="@ (e => Color_name(3))" />Син</li>
</ul>

@code{
private string cvyat = "pink";
private void Color_name(int c)
{
    //cvyat = c;
    switch (c)
    {
        case 1:
            cvyat = "red";
            break;
        case 2:
            cvyat = "green";
            break;
        case 3:
            cvyat = "blue";
            break;
        default:
            cvyat = "pink";
            break;
    }
}
}

```

Задача 2. Да се добавят един бутон и една текстова кутия. При натискане на бутона – той да добавя нов елемент (радио бутон) в списък, съответно с име “rb”+ поредния номер на радиобутона.

Допълнение: При избор на някой от радио бутоните от списъка в текстовото поле да се изпише кой е избран.

```

<button @onclick="up">Добави радиобутон</button>

<ul style="list-style-type:none">

@for(int i=1; i <= br; i++)
{
    int j = i;
    <li><input type="radio" name="rb" @onchange="@ (e=>izbran_nomer(j))" />@i</li>
}
</ul>
<input type="text" @bind="izbran" />

@code {
private int br = 0, izbran;

private void izbran_nomer(int n)
{
    izbran = n+1;
}

private void up()
{
    br++;
}
}

```

Задача 3. Да се добавят към проекта 4 изображения за 4-те сезона. При избор на конкретен радиобутон от списък с радиобутони (пролет, лято, есен, зима), да се показва изображението, съответстващо на избрания сезон.

Подсказка: Преименуване на изображенията със сходни имена, различаващи се само по една цифра, която отговаря на индекса на избрания от списъка елемент.

```
.....  
<ul style="list-style-type:none">
```

```
    @for (int i = 0; i < sezoni.Length; i++)  
    {  
        int j = i;  
        <li><input type="radio" name="rb" @onchange="@ (e => izbran_nomer(j))" />@sezoni[i]</li>  
    }  
</ul>
```

```

```

```
@code {  
    private int br = 0, izbran;  
    private string filename="";  
    private string[] sezoni = { "пролет", "лято", "есен", "зима"};  
  
    private void izbran_nomer(int n)  
    {  
        izbran = n+1;  
        filename = "image" + (n+1) + ".jpg";  
    }  
}
```

Задача 4. В страницата да се добави падащ списък с имена на градове (*Text*) и техните пощенски кодове (*Value*). Задачата да се реши по два начина - с използване на два списъка (*List*) и на колекция *Dictionary*.

Допълнение: Да се добави нова текстова кутия. При избор на елемент от падащия списък, избраният елемент да се визуализира в нея.

```
@page "/list"
```

```
@rendermode InteractiveServer
```

```
<h3>Областни градове</h3>
```

```
<select @bind="choise">  
    <option value="0">Изберете областен град</option>  
    @for (int i=0; i<grad.Count; i++)  
    {  
        <option value="@pk[i]">@grad[i]</option>  
    }  
</select>
```

```
<select @bind="choise">  
    <option value="0">Изберете областен град</option>  
    @for (int i = 0; i < oblasti.Count; i++)  
    {  
        <option value="@oblasti.ElementAt(i).Key">@oblasti.ElementAt(i).Value</option>  
    }  
</select>
```

```
<span>@choise</span>
```

```
<input type="text" @bind="choise" />
```

```
@code {
    private string? choose;

    private List<string> grad = new List<string> { "Варна", "София", "Пловдив" };
    private List<string> pk = new List<string> { "9000", "1000", "2000" };

    private Dictionary<string, string> oblasti = new Dictionary<string, string>
    {
        {"9000", "Варна"},
        {"1000", "София"},
        {"2000", "Пловдив"}
    };
}
```

Задача 5. Да се добавят текстова кутия, хипервръзка и бутон. Да се направи така, че при кликване върху бутона да се променя хиперлинк така, че той да сочи адреса, който е написан в текстовата кутия и при кликване да се отваря в нов прозорец.

Допълнение: Да се добавят две текстови кутии и бутон. Бутонът да добавя съдържанието на кутиите като нов елемент (заглавие и линк) към списък от линкове, който да се изобразява отдолу.

Елементите от списъка да се извеждат и в таблица.

```
.....
<input type="text" @bind="adres" @bind:after="napravi_adres" />
<a href="@adres" > @adres </a>
```

```
@code {

    private string adres = "";

    private void napravi_adres()
    {
        adres = "http://" + adres;
    }
}
```

Упражнение 12

Задача 1. Да се добавят 2 текстови кутии, падащ списък (който първоначално е скрит) и 2 бутона. При кликване на първия бутон да се прави проверка дали текстовите кутии са празни и дали въведения текст е вече въведен в падащия списък, ако не са празни и елементът го няма - да се добави съдържанието на текстовите кутии съответно като текст и стойност на нов елемент в списъка - например превозно средство и базова цена (да се ползва *Dictionary*). При кликване на втория бутон да се покаже съдържанието на падащия списък. Списъкът да е отворен - да се виждат всички елементи.

Допълнение: Съдържанието на *Dictionary-то* да се изведе в таблица.

```
.....
ПК: <input type="text" @bind="pk" />
<br />
Град: <input type="text" @bind="grad" />
<br />
<button @onclick="dobavi">Добави</button>

<select @bind="izbran_pk" @bind:after="check" size="@ (gradove.Count+1)"> //отворен списък

    <option value="0" disabled selected>--Избери--</option>
    @*
```

```

        @for(int i=0; i< gradove.Count; i++)
        {
            <option value="@gradove.ElementAt(i).Key">@gradove.ElementAt(i).Value</option>
        }
    *@

    @foreach(var item in gradove)
    {
        <option value="@item.Key">@item.Value</option>
    }
</select>

<span style="visibility:@show"> Резултат: @info</span>

<p>Градове:</p>
<table style="border-style:solid">
    <tr>
        <td>Номер</td>
        <td>ПК</td>
        <td>Град</td>
    </tr>
    @{ int i=1;}
    @foreach(var item in gradove)
    {
        <tr>
            <td>@i</td>
            <td>@item.Key</td>
            <td>@item.Value</td>
        </tr>
        i++;
    }
</table>

@code {
    Dictionary<string, string> gradove = new Dictionary<string, string>();
    private string? pk, grad, izbran_pk, info, show="hidden";
    private void dobavi()
    {
        if (!String.IsNullOrEmpty(pk) && !String.IsNullOrEmpty(grad) && !gradove.ContainsKey(pk))
        {
            gradove.Add(pk, grad);
        }
    }
    private void check()
    {
        if(izbran_pk != "0")
        {
            info = "Избраният град е с ПК " + izbran_pk;
        }
        else
        {
            info = "Не сте избрали град";
        }
        show = "visible";
    }
}

```

Задача 2. Изчисляване на месечна застрахователна вноска

Добавете нова страница, в която потребителят да може да изчисли месечната си застрахователна вноска, като избере:

- Тип превозно средство – напр. лека кола, мотоциклет, бус, камион - списък
- Период на застраховката – чрез радиобутони:

- o 6 месеца
 - o 12 месеца
- Допълнителна защита – чрез чекбоксове:
 - пътна помощ;
 - заместващ автомобил;
 - пожар и кражба.

След избор на съответните стойности и натискане на бутон „Изчисли“, програмата изчислява: общата застрахователна сума и месечната вноска.

Базови цени за година според превозното средство:

Лека кола – 600 евро
 Мотоциклет – 360 евро
 Бус – 780 евро
 Камион – 1200 евро

Такси за допълнителни услуги:

Пътна помощ: +100 евро
 Заместващ автомобил: +150 евро
 Пожар и кражба: +200 евро

@page "/zad2"

@rendermode InteractiveServer

<h3>Застраховки</h3>

```
<select @bind="cena_mps" size="@mps.Count">
  @for(int i=0; i < mps.Count; i++)
  {
    <option value="@mps.ElementAt(i).Value">@mps.ElementAt(i).Key</option>
  }
</select>
```

Изберете период:

```
<ul>
  <li><input type="radio" name="period" @onchange="@ (e=>period(6))" />6 месеца</li>
  <li><input type="radio" name="period" @onchange="@ (e=>period(12))" />12 месеца</li>
</ul>
```

Защита:

```
<ul>
  @for(int i=0; i < protect.Count; i++)
  {
    int j = i;
    <li><input type="checkbox" @bind="@check[j]" />@protect[i]</li>
  }
</ul>
```

<button @onclick="izchisli">Пресметни</button>

<p style="visibility:@show">Дължима сума: @suma евро. Месечна вноска: @vnoska евро.</p>

<p style="visibility:@show_msg">@msg</p>

```
@code {
  private Dictionary<string, int> mps = new Dictionary<string, int>
  {
    {"лека кола", 600 },
    {"камион", 1200},
    {"микробус", 780 },
    {"мотоциклет", 360}
  };
  private string? cena_mps, show = "hidden", show_msg = "hidden", msg;
  private int meseci = -1, suma, dobavki;
```



```

private double vnoska=0;

private List<string> protect = new List<string> { "пътна помощ", "заместващ автомобил", "пожар и кражба" };
private List<int> price = new List<int> { 100,150,200 };
private List<bool> check = new List<bool> { false,false, false};

private void period(int n)
{
    meseci = n;
    show_msg = "hidden";
}

private void izchisli()
{
    dobavki = 0;
    for(int i=0; i<check.Count; i++)
    {
        if (check[i])
        {
            dobavki += price[i];
        }
    }

    suma = int.Parse(cena_mps) + dobavki;
    if (meseci > -1)
    {
        vnoska = suma / meseci;
        show = "visible";
    }
    else
    {
        msg = "Изберете период";
        show_msg = "visible";
    }
}
}
}

```

Упражнение 13

Работа с SQLite база от данни

Задача 1. Към проекта да се добави поддръжка за работа с **SQLite** база от данни:

Project -> Manage NuGet Packages -> Browse и намиране и инсталиране на:

- Microsoft.EntityFrameworkCore
- Microsoft.EntityFrameworkCore.Sqlite

Задача 2. Да се създаде база от данни с име **auto** и в нея таблица **mps** с полета: **MpsId** (int, primary key), **MpsName** (string), **Tax**(float)

Десен бутон към проекта -> Add -> New Item -> Class , който да се казва Mps и в него:

```
using Microsoft.EntityFrameworkCore;
```

```
namespace . . . . .
```

```
[PrimaryKey(nameof(MpsId))]
```

```

public class Mps // класът (моделът), който описва полетата в таблицата
{
    public int MpsId { get; set; }
    public string? MpsName { get; set; }
    public float Tax { get; set; }
}

public class AutoContext : DbContext // връзката с базата от данни
//AutoContext е наше име на клас, който наследява класа DbContext (специален клас от Entity
//Framework Core), който представлява връзката между C# кода и базата от данни. Чрез него могат да
//се четат, записват, създават и обновяват данните в таблиците.
{
    public DbSet<Mps> mps{ get; set; } //представя таблицата mps
// DbSet<Mps> осигурява възможности за добавяне на записи: mps.Add(...), търсене: mps.Find(...), зареждане на
// списък: mps.ToList()

    protected override void OnConfiguring(DbContextOptionsBuilder options)
    => options.UseSqlite("Data Source=auto.db"); //посочваме името на базата от данни
}

```

В Program.cs преди app.Run да се добави:

```

using (var context = new AutoContext())
{
    context.Database.EnsureCreated(); //ако базата от данни не съществува, ще бъде създадена
}

```

Задача 3. В нова страница да се добавят 2 текстови кутии и бутон за вмъкване на нов запис в таблица *mps*.

```

.....
Вид МПС:
<input type="text" @bind="vid_mps" />
<br />
Цена:
<input type="text" @bind="bazova_cena" />
<br />
<button @onclick="dobavi_db">Добави в БД</button>

```

```

@code {
    AutoContext auto_data = new AutoContext(); //връзката между C# кода и базата от данни
    .....
    private void dobavi_db()
    {
        Mps m = new Mps();
        m.MpsName = vid_mps;
        m.Tax = bazova_cena;
        auto_db.mps.Add(m);
        auto_db.SaveChanges(); //при промяна да не забравяме auto_data.SaveChanges();
    }
}

```

Задача 4. Да се добави падащ списък, в който да се извеждат извеждат записите от таблица *mps*.

Допълнение: При избор на елемент от списъка в текстова кутия да се показва неговия код (*MpsId*).

```

.....
<select @bind="izbrano_id">
    @foreach(Mps i in auto_data.mps)
    {
        <option value="@i.MpsId">@i.MpsName</option>
    }

```

```
</select>
```

```
<input type="text" @bind="izbrano_id" />
```

Задача 5. Да се добави нова текстова кутия. При избор на МПС от списъка в кутията да се изписва базовата му цена.